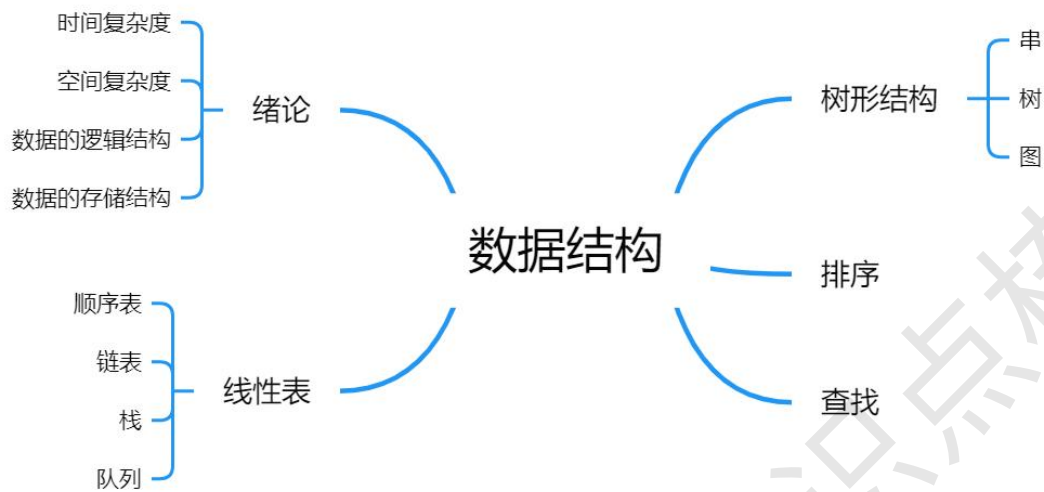


计算机类推免面试知识点梳理

目录

1. 数据结构	1
1.1 绪论	1
1.2 线性表	2
1.3 树形结构	4
1.4 排序	6
1.5 查找	7
2. 计算机组成原理	9
2.1 计算机系统概述	9
2.2 计算机的总线与 I/O 设备	10
2.3 计算机的存储器	11
2.4 计算机的 CPU	13
3. 计算机的操作系统	15
3.1 操作系统概述	15
3.2 进程管理	16
3.3 存储管理	18
3.4 文件管理和设备管理	20
4. 计算机网络	22
4.1 计算机网络概述	22
4.2 物理层	23
4.3 数据链路层	24
4.4 网络层	26
4.5 传输层	27
4.6 应用层	29
5. 数据库原理	31
5.1 绪论	31
5.2 事务	32
5.3 SQL 语句	34
5.4 关系数据理论与数据库设计	36

1. 数据结构



1.1 绪论

1.1.1 时间复杂度

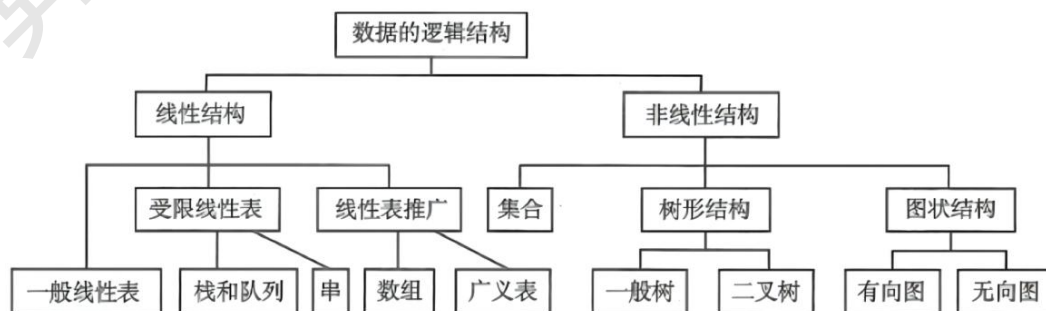
一个语句的频度是指该语句在算法中被重复执行的次数。算法中所有语句的频度之和记为 $O(n)$ ，它是该算法问题规模 n 的函数，时间复杂度主要分析 $O(n)$ 的数量级。

1.1.2 空间复杂度

算法的空间复杂度 $S(n)$ 定义为该算法所耗费的存储空间，它是问题规模 n 的函数。

1.1.3 数据的逻辑结构

逻辑结构指的是数据元素之间逻辑关系，与数据的存储结构无关，是独立于计算机的，以下是分类图。



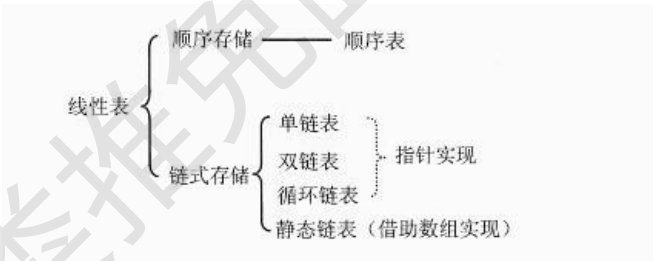
1.1.4 数据的存储结构

数据的存储结构是指数据的逻辑结构在计算机中的存储方式。又称物理结构。主要包括以下 4 种：顺序存储、链式存储、索引存储、散列存储。其优缺点比较如下：

存储结构	优点	缺点
顺序结构	随机存取，每个元素占用最少的存储空间	只能使用相邻的一整块存储单元，因此可能产生较多的外部碎片
链式结构	无碎片现象，能充分利用所有存储单元	每个元素因存储指针而占用额外的存储空间，且只能实现顺序存取
索引结构	检索速度快	附加的索引表额外占用存储空间。索引表需要随数据改变修改，因而会花费较多的时间。
散列存储	检索、增加和删除结点的操作都很快	若散列函数不好，则可能出现元素存储单元的冲突，因而增加时间和空间开销

1.2 线性表

线性表是一种连续或间断存储的数组，这里的连续和间断是针对物理内存空间中线性表元素之间是否连续，其中连续数组对应内置数组的实现方式，间断数组对应的是指针的实现方式，即链表实现。



1.2.1 顺序表和链表的比较

	顺序表	链表
存取（读写）方式	顺序存取、随机存取	顺序存取
逻辑结构与物理结构	逻辑上相邻的元素，对应的物理存储位置也相邻	逻辑上相邻的元素，物理存储位置则不一定相邻
查找、插入和删除操作	对于按值查找，无序的时间复杂度为 $O(n)$ ，有序则为 $O(\log_2 n)$ 。 对于按序号查找，时间复杂度为 $O(1)$ 。 顺序表的插入、删除操作，平均需要移动半个表长的	对于按值查找，时间复杂度为 $O(n)$ 。 对于按序号查找，链表的平均时间复杂度为 $O(n)$ 。 链表的插入、删除操作，只需修改相关结点的指针域。由于链表的每个结点都带有指针域，故而

	元素	存储密度不够大。
空间分配	静态存储分配情形下,一旦存储空间装满就不能扩充。动态存储分配虽然存储空间可以扩充,但操作效率低	结点空间只在需要时申请分配,只要内存有空间就可以分配,操作灵活、高效。

常见问题:

头指针和头结点的区别?

答: 头指针:是指向第一个节点存储位置的指针,具有标识作用,头指针是链表的必要元素,无论链表是否为空,头指针都存在。

头结点:是放在第一个元素节点之前,便于在第一个元素节点之前进行插入和删除的操作,头结点不是链表的必须元素,可有可无,头结点的数据域也可以不存储任何信息。

1.2.2 栈与队列的比较

栈和队列都是一种**线性表**。两种实现方式,一种是用内置数组实现,一种是以链表实现。

对于进入队列的元素按“**先进先出**”的规则,在表头进行删除在表尾进行插入。由于队列要进行频繁的插入和删除,一般为了高效,选择用**定长数组**来存储队列元素,在对队列进行操作之前要**判断队列是否为空或是否已满**。如果想要动态长度也可以用链表来存储队列,需要记住队头和队尾指针的地址。

对于插入到栈的元素按“**后进先出**”的规则,插入和删除操作都在栈顶进行。与队列类似一般用**定长数组**存储栈元素。由于进栈和出栈都是在栈顶进行,因此要有一个 **size 变量**来记录当前栈的大小。



常见问题

如何区分循环队列是队空还是队满?

答: 方法一: **牺牲一个单元**来区分队空和队满, 这个时候 $Q.rear == Q.front$ 才是队满标志。

方法二: 类型中增设表示**元素个数**的数据成员。这样, 队空的条件为 $Q.size == 0$; 队满的条件为 $Q.size == MaxSize$ 。

1.3 树形结构

1.3.1 串

串是零个或多个字符组成的有限序列，如 $S = \text{'iPhone 11 Pro Max?'}$ 。模式匹配为串的常见考点，算法包括朴素模式匹配算法、KMP 算法等。

(1) 朴素模式匹配算法

匹配思想是：从主串的第一个字符起，与子串的第一个字符比较，相等则继续比较；不等则从主串的下一个位置起，继续和子串开始比较，直到最后看是否匹配成功。

算法最坏时间复杂度 = $O(nm)$ ，最好时间复杂度 = $O(n)$

(2) KMP 算法

匹配思想为：KMP 算法从分析模式本身的结构着手，如果已匹配相等的前缀序列中有某个后缀正好是模式的前缀，那么就可以将模式向后滑动到与这些相等字符对齐的位置，主串 i 指针无须回溯，并继续从该位置开始进行比较。而模式向后滑动位数的计算仅与模式本身的结构有关，与主串无关。

最好时间复杂度 = $O(n)$ 。通常，没有指出最坏或者最好的情况下，默认为最坏。

1.3.2 树

树是非线性结构，其元素之间有明显的层次关系。在树的结构中，每个节点都只有一个前驱为父节点，没有前驱节点为树的根节点，简称为树的根；每个节点可以有多个后继称为节点的子节点，没有后继的节点称为叶子节点。

二叉树是另一种树形结构，其特点是每个结点至多只有两棵子树，并且二叉树的子树有左右之分。

常见考点为二叉树的遍历、线索二叉树、树的存储。

(1) 二叉树的遍历

类型	思想
先序遍历	二叉树为空：不用操作 二叉树非空：访问根节点→先序遍历左子树→先序遍历右子树
中序遍历	二叉树为空：不用操作 二叉树非空：先序遍历左子树→访问根节点→先序遍历右子树
后序遍历	二叉树为空：不用操作 二叉树非空：先序遍历左子树→先序遍历右子树→访问根节点
层序遍历	初始化一个辅助队列→根节点入队→ 若队列非空，则队头结点出队，访问该结点，依次将其左、右孩子插入队尾（如果有的话）→重复以上操作直至队列为空

(2) 线索二叉树

对于 n 个结点的二叉树，在二叉链存储结构中有 $n+1$ 个空链域，利用这些空链域存放在某种遍历次序下该结点的前驱结点和后继结点的指针，这些指针称为线索，加上线索的二叉树称为线索二叉树。

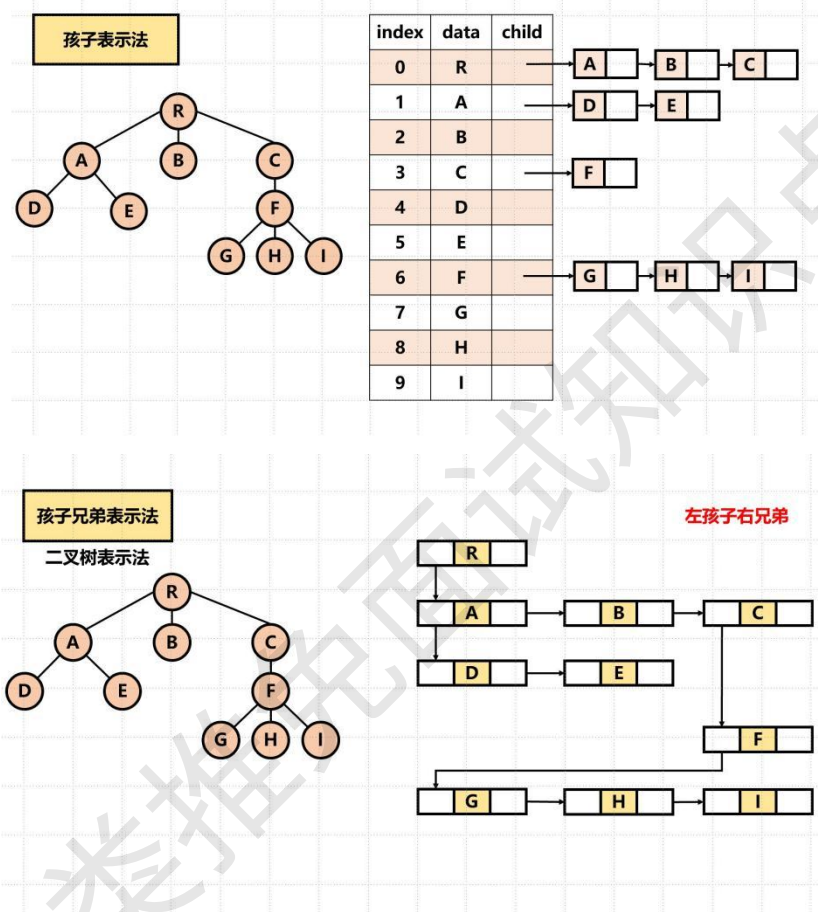
(3) 树的存储

树的存储方式包括三种：双亲表示法、孩子表示法、孩子兄弟表示法。

双亲表示法采用一组连续空间来存储每个结点，同时在每个结点中增设一个伪指针，指示其双亲结点在数组中的位置。

孩子表示法是将每个结点的孩子结点都用单链表链接起来形成一个线性结构，此时 n 个结点就有 n 个孩子链表（叶子结点的孩子链表为空表）。

孩子兄弟表示法又称二叉树表示法，即以二叉链表作为树的存储结构。孩子兄弟表示法使每个结点包括三部分内容：结点值、指向结点第一个孩子结点的指针，及指向结点下一个兄弟结点的指针（沿此域可以找到结点的所有兄弟结点）。



1.3.3 图

图是由顶点的有穷非空集合的顶点之间边的集合组成，通常表示为： $G(V, E)$ 。其中 G 表示为一个图， V 是图 G 中顶点的集合， E 为图 G 的边的集合。

常见考点为图的遍历，包括两种：深度优先搜索、广度优先搜索。

(1) **深度优先搜索 (DFS)**：深度优先搜索 (Depth First Search - DFS) 遍历类似树的先序遍历，是树的先序遍历的推广。

设初始状态时图中的所有顶点未被访问，则：

- ①从图中某个顶点 v_i 出发，访问 v_i ；然后找到 v_i 的一个邻接顶点 v_{i1} ；
- ②从 v_{i1} 出发，深度优先搜索访问和 v_{i1} 相邻接且未被访问的所有顶点；
- ③转①，直到和 v_i 相邻接的所有顶点都被访问为止
- ④继续选取图中未被访问顶点 v_j 作为起始顶点，转①，直到图中所有顶点都被访问为止。

(2) **广度优先搜索(BFS)**: 广度优先搜索(Breadth First Search - BFS)遍历类似树的按层次遍历的过程。

设初始状态时图中的所有顶点未被访问, 则:

- ①从图中某个顶点 v_i 出发, 访问 v_i ;
- ②访问 v_i 的所有相邻接且未被访问的所有顶点 $v_{i1}, v_{i2}, \dots, v_{im}$;
- ③以 $v_{i1}, v_{i2}, \dots, v_{im}$ 的次序, 以 $v_{ij}(1 \leq j \leq m)$ 依此作为 v_i , 转(1);
- ④继续选取图中未被访问顶点 v_k 作为起始顶点, 转①, 直到图中所有顶点都被访问为止。

1.4 排序

排序就是一系列数据, 按照某个关键字(例如: 销量, 价格), 进行递增或者递减的顺序排列起来。

1.4.1 排序的分类

按照排序过程中所依据的原则的不同可以分为:

- (1) 插入排序: 直接插入排序、希尔排序
- (2) 交换排序: 冒泡排序、快速排序
- (3) 选择排序: 简单选择排序、堆排序
- (4) 归并排序
- (5) 基数排序

1.4.2 排序算法比较

类别	排序方法	时间复杂度			空间复杂度	稳定性
		平均情况	最好情况	最坏情况	辅助存储	
插入排序	直接插入	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
	Shell 排序	$O(n^{1.5})$	$O(n)$	$O(n^2)$	$O(1)$	不稳定
选择排序	直接选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
	堆排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(1)$	不稳定
交换排序	冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
	快速排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n^2)$	$O(n \log_2 n)$	不稳定
归并排序		$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n)$	稳定
基数排序		$O(d(r+n))$	$O(d(r+n))$	$O(d(r+n))$	$O(r)$	稳定

注: 基数排序的复杂度中, r 代表关键字的基数, d 代表长度, n 代表关键字的个数

从平均情况看: 堆排序、归并排序、快速排序胜过希尔排序。

从最好情况看: 冒泡排序和直接插入排序更胜一筹。

从最差情况看: 堆排序和归并排序强过快速排序。

虽然直接插入排序和冒泡排序速度比较慢, 但是当初始序列整体或局部有序是, 这两种算法的效率比较高。当初始序列整体或局部有序时, 快速排序算法效

率会下降。当排序序列较小且不要求稳定性是，直接排序效率较好；要求稳定性时，冒泡排序法效率较好。

1.5 查找

查找的过程是将给定的 K 值与文件中各记录的关键词比较。用比较次数的平均值评估算法的优劣，即平均查找长度（ASL）。显然，ASL 值越小，时间效率越高。

1.5.1 查找的分类

- (1) 静态查找算法：顺序查找、折半查找、分块查找
- (2) 动态查找算法：二叉排序法、平衡二叉树、红黑树、B 树、键树
- (3) 哈希表

1.5.2 常见查找算法

(1) 顺序查找

把待查关键字 key 放入哨兵位置 ($i=0$)，再从后往前依次把表中元素和 key 比较，如果返回值为 0 则查找失败，表中没有这个 key 值，如果返回值为元素的位置 i ($i \neq 0$) 则查找成功。时间效率为 $O(n)$ 。

(2) 折半查找

要求查找表为顺序存储结构并且有序，若关键字在表中，则返回关键字的位置，若关键字不在表中时停止查找的典型标志是：查找范围的上界 $\leq$ 查找范围的下界。

(3) 分块查找

先把查找表分为若干子表，要求每个子表的元素都要比他后面的子表的元素小，也就是保证块间是有序的（但是子表内不一定有序），把各子表中的最大关键字构成一张索引表，表中还包含各子表的起始地址。

(4) 二叉排序树

二叉排序树的定义为：或者是一棵空树，或者是一棵具有如下特点：如果该树有左子树，则其左子树的所有节点值小于根的值；若该树有右子树，则其右子树的所有节点值均大于根的值；其左右子树也分别为二叉排序树。在查找时可以进行动态的插入，插入节点要符合二叉排序树的定义，这也是动态查找和静态查找的区别，静态查找不能进行动态插入。

(5) 平衡二叉树

平衡二叉树又称为 AVL 树，它或者是一棵空树或者具有如下特点：他的左子树和右子树的高度差的绝对值不能大于 1，且他的左右子树也都是平衡二叉树。如果再一个平衡二叉树中插入一个节点可能造成失衡，这时就要进行树结构的调整，即平衡旋转。包括 4 中情况：

在左子树的左子树上插入节点时向右进行单向旋转；

在右子树的右子树上插入节点时向左进行单向旋转；

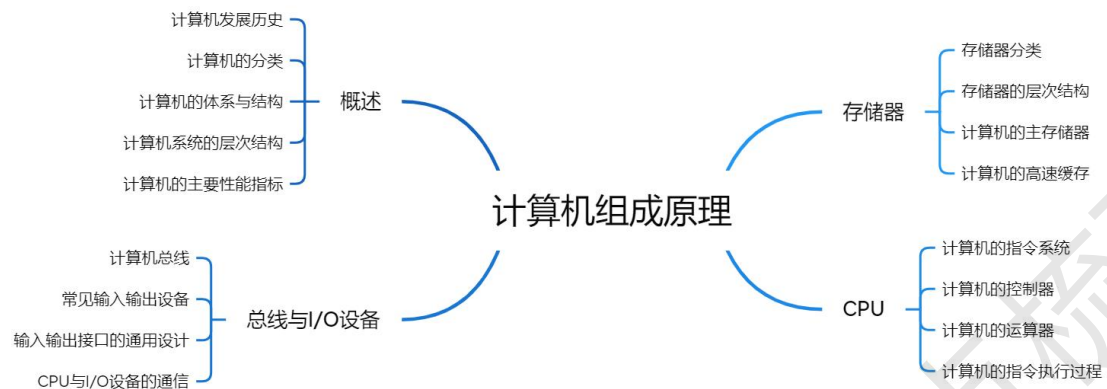
在左子树的右子树插入节点时先向左旋转再向右旋转；

在右子树的左子树插入节点时先向右旋转再向左旋转。

(6) 红黑树

红黑树是平衡查找树的改进。红黑树的思想是对 2-3 查找树进行编码，尤其是对 2-3 查找树中的 3-nodes 节点添加额外的信息。红黑树中将节点之间的链接分为两种不同类型，红色链接，他用来链接两个 2-nodes 节点来表示一个 3-nodes 节点。黑色链接用来链接普通的 2-3 节点。特别的，使用红色链接的两个 2-nodes 来表示一个 3-nodes 节点，并且向左倾斜，即一个 2-node 是另一个 2-node 的左子节点。

2. 计算机组成原理



2.1 计算机系统概述

2.1.1 计算机的发展历史

- (1) 第一阶段（1946-1956）：电子管计算机 特点：集成度低，体积大，功耗高，运行速度慢，操作复杂。
- (2) 第二阶段（1956-1964）：晶体管计算机 特点：相对电子管计算机，体积小，速度快，功耗低，可靠性高，配备显示器。
- (3) 第三阶段（1964-1970）：集成电路计算机 特点：操作系统诞生。
- (4) 第四阶段（1970-至今）：超大规模集成电路计算机 特点：集成度高，速度快，体积小，价格低，用途广泛。
- (5) 第五阶段（future）：量子计算机……

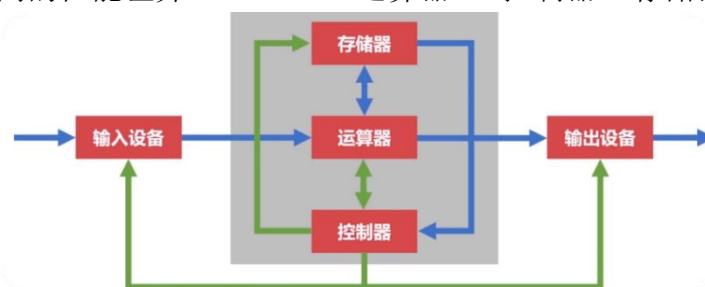
2.1.2 计算机的分类

超级计算机、大型计算机、迷你计算机（普通服务器）、工作站、微型计算机（个人计算机）。

2.1.3 计算机的体系与结构

(1) 冯·诺伊曼体系：将程序指令和数据一起存储的计算机设计概念结构，由存储器、控制器、运算器、输入设备、输出设备五个部分组成。

(2) 现代计算机的结构：以存储器为核心，解决冯·诺伊曼体系瓶颈问题（CPU与存储设备之间的性能差异）。CPU = 运算器 + 控制器 + 存储器



2.1.4 计算机系统的层次结构

从下到上依次为：微程序设计级、一般机器级、操作系统级、汇编语言级、高级语言级。

(1) 硬件：

第 1 级：微程序机器层 M0（微指令系统），由硬件直接执行微指令。

第 2 级：传统机器 M1（机器语言机器），用微程序解释机器指令。

(2) 软件：

第 3 级：虚拟机器 M2（操作系统机器），用机器语言解释操作系统。

第 4 级：汇编语言 M3（汇编语言机器），用汇编程序翻译成机器语言程序。

第 5 级：虚拟机器 M4（高级语言机器），用编译程序翻译成汇编语言程序。

2.1.5 计算机的主要性能指标

(1) 机器字长：CPU 一次能处理的二进制数据的位数。

(2) 数据通路带宽：数据总线一次所能并行传送信息的位数。

(3) 主存容量：主存储器所能存储信息的最大容量

(4) 运算速度

①吞吐量：指系统在单位时间内处理请求的数量。

②响应时间：指从用户向计算机发送一个请求，到系统对该请求做出响应并获得所需结果的等待时间。

③主频：机器内部时钟的频率。

CPU 时钟周期：主频的倒数，它是 CPU 中最小的时间单位，每个动作至少需要 1 个时钟周期。

④CPI(Clock cycle Per Instruction)：即执行一条指令所需的时钟周期数。

2.2 计算机的总线与 I/O 设备

2.2.1 计算机的总线

(1) 概述：计算机系统各部件之间进行信息传输的公共通道。(即：多个部件共享传输部件)

(2) 作用：解决不同设备之间的通信问题。

(3) 特点：某一时刻只能有一路信息在总线上传输，即：分时使用。

(4) 分类：片内总线（高集成度内部的信息传输线）、系统总线（细分为：数据总线、地址总线、控制总线，是 CPU、主内存、IO 设备、各组件之间的信息传输线）

(5) 总线的仲裁：目的是解决总线使用权的冲突问题。包括三种方法：链式查询、计时器定时查询、独立请求。

	链式查询	计时器定时查询	独立请求
控制线数	3	$\lceil \log_2 n \rceil + 2$	$2n + 1$
总线忙 1、总线请求 1、	总线忙 1、总线请求 1、	总线忙 1、总线请求 1、	总线忙 1、总线请求 n、

	总线同意 1	总线同意 $\lceil \log_2 n \rceil$	总线同意 n
优点	优先级固定、结构简单、容易扩充设备	优先级灵活（介于两者之间）	响应速度快、优先次序灵活
缺点	对电路故障不敏感、优先级不灵活（优先级别低的设备难以获得请求）	对电路故障敏感介于两者之间、控制线路多、控制复杂	控制线路多、控制复杂

2.2.2 常见的输入输出设备

- (1) 字符输入设备：键盘
- (2) 图形输入设备：鼠标、数位板、扫描仪
- (3) 图像输出设备：显示器、打印机、投影仪

2.2.3 输入输出接口的通用设计

- (1) 数据线：I/O 设备与主机进行数据交换的传送线（单向&双向）。
- (2) 状态线：I/O 设备状态向主机报告的信号线。
- (3) 命令线：CPU 向 I/O 设备发送命令（读写信号、启动停止信号）的信号线。
- (4) 设备选择线：主机选择 I/O 设备进行操作的信号线。

2.2.4 CPU 与 I/O 设备的通信

(1) 程序中断：提供低速设备通知 CPU 的一种异步的方式，CPU 可以在高速运转的同时兼顾低速设备的响应。

(2) 直接存储器访问（DMA）：在主存和 I/O 设备之间建立独立的总线连接。DMA 和主存交换数据时，通常采取停止 CPU 访问主存、周期挪用（窃取）、DMA 和 CPU 交替访问三种方式，解决争用主存冲突的问题。

常见问题

同步通信与异步通信辨析：

答：(1) 同步通信，由统一的时钟控制，控制方式简单，灵活性差，当系统中的各部件工作速度差异大的时候，总线工作效率明显下降，适用于差别不大的场合。

(2) 异步通信，指没有统一时钟控制的通信，部件之间采用应答方式进行联系，控制方式较同步方式，灵活度高，当系统中各部件工作速度差异大的时候，有利于提高总线工作效率。

2.3 计算机的存储器

2.3.1 存储器分类

- (1) 按照存储介质：半导体存储器（内存、U 盘、固态硬盘）、磁存储器（磁带、磁盘）

(2) 按照存取方式：随机存储器 RAM（随机读取，与位置无关）、串行存储器（按顺序查找，与位置有关）、只读存储器 ROM（只读不写）

2.3.2 存储器的层次结构

(1) 缓存-主存层次：局部性原理，在 CPU 与主存之间增加一层速度快容量小的 Cache，解决主存速度不足的问题。

(2) 主存-辅存层次：局部性原理，主存之外增加辅助存储器，解决主存容量不足的问题。

(3) 局部性原理：是指 CPU 访问存储器时，无论是存取指令还是存取数据，所访问的存储单元都趋于聚集在一个较小的连续区域中。



2.3.3 计算机的主存储器

(1) 主存储器包括：随机存储器 RAM(静态 RAM 和动态 RAM)、只读存储器等。

(2) 主要技术指标：存储容量、存取速度和存储带宽。

(3) 主存储器相关算法：

存储容量 = 存储单元个数 × 存储字长

存储容量(字节) = 存储单元个数 × 存储字长 / 8

存储周期 = 存储时间 + 恢复时间

2.3.4 计算机的高速缓存 (Cache)

(1) 工作原理：命中率是衡量缓存的重要性能指标，理论上 CPU 每次都能从高速缓存存取数据的时候，命中率为 1。

(2) 高速缓存的替换时间：当缓存没有数据，需要从主装载入数据的时候。

(3) 高速缓存的替换策略：随机算法、先进先出算法 (FIFO)、最不经常使用算法 (LFU)、最近最少使用算法 (LRU)。

高速缓存的计算：

$$\text{命中率: } h = \frac{N_c}{N_c + N_m}$$

$$\text{Cache-主存系统的平均访问时间: } t_a = ht_c + (1 - h)t_m$$

$$\text{访问效率: } e = \frac{t_c}{t_a} \times 100\% = \frac{t_c}{ht_c + (1 - h)t_m} \times 100\%$$

h —命中率; $1-h$ —未命中率;
 N_c —访问 Cache 的总命中次数;
 N_m —访问主存的总次数 (不命中次数);
 t_a —Cache-主存系统的平均访问时间;
 t_c —命中时的 Cache 访问时间;
 t_m —未命中时的主存访问时间;
 e —访问效率。

2.4 计算机的 CPU

2.4.1 计算机的指令系统

(1) 机器指令的形式: 操作码 (指明指令所要完成的操作)+地址码 (给出操作数或操作数的地址)。

(2) 机器指令的操作类型: 数据传输、算术逻辑操作、移位操作、控制指令。

(3) 机器指令的寻址方式: 指令寻址 (顺序寻址+跳跃寻址)、数据寻址 (**立即寻址** (速度快)+**直接寻址** (寻找操作数简单)+**间接寻址** (寻址范围大, 速度慢))。

2.4.2 计算机的控制器

(1) 作用: 控制器是协调和控制计算机运行的。

(2) 组成: 程序计数器 (存储下一条指令的地址)、**时序发生器** (发送时序脉冲)、**指令译码器** (控制器的主要部件之一, 翻译操作码+地址码)、**指令寄存器** (控制器的主要部件之一, 从主存或缓存存取计算机指令)、主存地址寄存器 (保存当前 CPU 正要访问的内存地址单元)、主存数据寄存器 (保存当前 CPU 正要读或写的主存数据)、通用寄存器 (比一般专用寄存器大, 可以暂时存放或传送数据或指令, 可保存 ALU 的运算中间结果)。

2.4.3 计算机的运算器

(1) 作用: 进行数据运算加工。

(2) 组成: 数据缓冲器 (输入缓冲暂时存放外设送过来的数据, 输出缓冲暂时存放送往外设的数据)、**ALU** (算术逻辑运算)、状态字及寄存器 (存放运算状态和运算控制信息)、通用寄存器 (比一般专用寄存器大, 可以暂时存放或传送数据或指令, 可保存 ALU 的运算中间结果)。

2.4.4 计算机的指令执行过程

(1) 指令执行过程: 取指令-分析指令-执行指令。



(2) CPU 的流水线设计：在每一个时钟周期启动一条指令，尽量让多条指令同时运行，但各自处在不同的执行步骤中。解决因运算器和控制器不能同时工作，导致 CPU 的综合利用率不高的问题。CPU 的流水线设计可以使 CPU 利用率达到原来的三倍。

流水线的时钟周期： $T = \max\{\text{各段时间}\} = \Delta t$

n 条指令需要的时间： $m\Delta t + (n-1)\Delta t$

吞吐率：

$$\text{最大吞吐率: } T_{p\max} = \frac{1}{\Delta t}$$

$$\text{实际吞吐率: } T_{p\max} = \frac{n}{m\Delta t + (n-1)\Delta t}$$

各段时间均为 Δt (取时钟周期)
完成 n 条指令
每条指令 m 段

加速比：指 m 段流水线的速度与等功能的非流水线的速度之比。

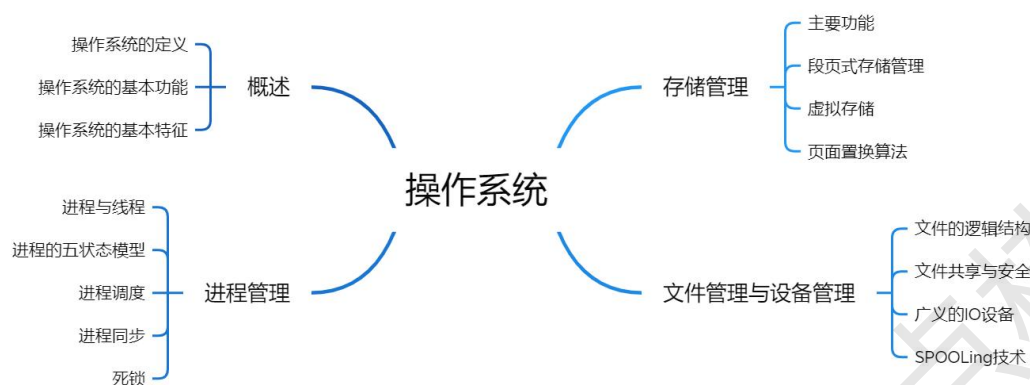
$$S_p = \frac{n \times \text{各段时间之和}}{(m+n-1) \times \Delta t}$$

常见问题

CPU 概述：

答：由运算器和控制器组成，功能包括：指令控制，操作控制，时间控制，数据加工，中断处理。

3. 计算机的操作系统



3.1 操作系统概述

3.1.1 操作系统的定义

操作系统是一个大型的程序系统，是核心系统软件。

3.1.2 操作系统的基本功能

- (1) **统一管理计算机资源**：处理器资源，IO 设备资源，存储器资源，文件资源等。
- (2) **实现了对计算机资源的抽象**：IO 设备管理软件提供读写接口，文件管理软件提供操作文件接口等。
- (3) **提供了用户与计算机之间的接口**：图像窗口形式，命令形式，系统调用形式等。



3.1.3 操作系统的基本特征

- (1) **并发**：指两个或多个事件在同一时间间隔内发生，微观上还是程序在分时地交替执行。

(2) **共享**：指系统中的资源可供内存中多个并发执行的进程共同使用，主要包括两个方式，一种为互斥共享方式，例如如打印机，在一段时间内只允许一个进程访问该资源；另一种为同时访问方式，例如磁盘设备。

(3) **虚拟**：指把一个物理上的实体变为若干个逻辑上的对应物，例如虚拟处理器，虚拟内存，虚拟外部设备。

(4) **异步**：在多道程序环境下，允许多个程序并发执行，但由于资源有限，进程的执行不是一贯到底，而是走走停停，以不可预知的速度向前推进。
其中，并发和共享是操作系统的最基本特征。

常见问题

操作系统的四个基本特征。解答见上文。

3.2 进程管理

3.2.1 进程与线程

(1) 线程（Thread）：是操作系统进行运行调度的最小单位。

(2) 进程（Process）：一个程序在给定活动空间和初始环境下，在一个处理机上的执行过程，是一个动态的概念。系统进行资源分配和调度的基本单位。

(3) 区别与联系：一个进程可以有一个或多个线程；线程包含在进程之中，是进程中实际运行工作的单位；进程的线程**共享进程资源**；一个进程可以并发多个线程，每个线程执行不同的任务。

	进程	线程
资源	资源分配的基本单位	不拥有资源
调度	独立调度的基本单元	独立调度的最小单位
系统开销	进程系统开销大	线程系统开销小
通信	进程 IPC	读写统一进程数据通信

3.2.2 进程的五状态模型

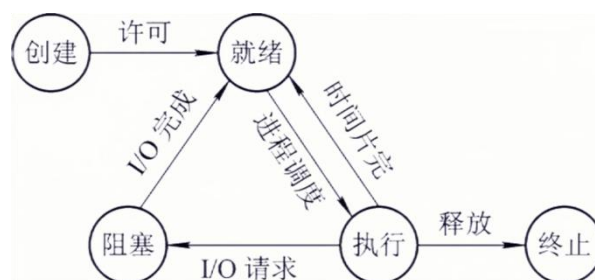
(1) 就绪状态：其它资源（进程控制块、内存、栈空间、堆空间等）都准备好、只差 CPU 的状态。

(2) 执行状态：进程获得 CPU，其程序正在执行。

(3) 阻塞状态：进程因某种原因放弃 CPU 的状态。

(4) 创建状态：创建进程时拥有 PCB 但其它资源尚未就绪。

(5) 终止状态：进程结束由系统清理或者归还 PCB 的状态。



3.2.3 进程调度

(1) 定义：计算机通过决策决定哪个就绪进程可以获得 **CPU 使用权**。

(2) 方法：抢占式调度、非抢占式调度。

	抢占式调度	非抢占式调度
系统开销	频繁切换、开销大	切换次数少、开销小
公平性	相对公平	不公平
应用	通用系统	专用系统

(3) **调度算法**：

①先来先服务算法：按照在就绪队列中的**先后顺序**执行。

②短进程优先调度算法：优先选择就绪队列中估计运行时间最短的进程，**不利于长作业进程的执行**。

③高优先权优先调度算法：进程附带优先权，优先选择权重高的进程，可以使得**紧迫的任务优先处理**。

④时间片轮转调度算法：按照 FIFO 的原则排列就绪进程，每次从队列头部取出待执行进程，分配一个时间片执行，是**相对公平**的调度算法，但是不能保证就是响应用户。

3.2.4 进程同步

(1) 临界资源与临界区：临界资源即一次仅允许一个进程使用的资源，临界区即对临界资源进行访问的代码。为了互斥访问临界资源，每个进程在进入临界区之前，需要先进行检查。

(2) **同步与互斥**：同步即多个进程按一定顺序执行；互斥即多个进程在同一时刻只有一个进程能进入临界区。同步是在对临界区互斥访问的基础上，通过其它机制来实现有序访问的。

(3) **同步机制遵循原则**：空闲让进、忙则等待、有限等待、让权等待。

经典的进程同步问题：生产者-消费者问题；哲学家进餐问题；读者-写者问题。

3.2.5 死锁

(1) 定义：死锁是指多个进程在运行过程中，因为争夺资源而造成的一种僵局，如果没有外力推进，处于僵局中的进程就无法继续执行。通俗来说，就是两个或多个进程被无限期地阻塞、相互等待的一种状态。

(2) 产生原因：竞争资源、进程推进顺序非法。

(3) **必要条件**：

①互斥条件：进程对所分配的资源进行排他性的使用。

②请求和保持条件：进程被阻塞的时候并不释放锁申请到的资源。

③不可剥夺条件：进程对于已经申请到的资源在使用完成之前不可被剥夺。

④环路等待条件：发生死锁的时候存在的一个 进程-资源 环形等待链。

(4) 处理方法

①预防死锁：破坏产生死锁的 4 个必要条件中的一个或者多个。实现起来比较简单，但是如果限制过于严格会降低系统资源利用率以及吞吐量。

②避免死锁：在资源的动态分配中，防止系统进入不安全状态(可能产生死锁的状态)。如银行家算法。

③检测死锁：允许系统运行过程中产生死锁，在死锁发生之后，采用一定的算法进行检测，并确定与死锁相关的资源和进程，采取相关方法清除检测到的死锁。但实现难度大。

④解除死锁：与死锁检测配合，将系统从死锁中解脱出来（撤销进程或者剥夺资源）。对检测到的和死锁相关的进程以及资源，通过撤销或者挂起的方式，释放一些资源并将其分配给处于阻塞状态的进程，使其转变为就绪态。实现难度大。

常见问题

1、线程同步的方式

答：（1）**临界区**：通过对多线程的串行化来访问公共资源或一段代码，速度快，适合控制数据访问。

（2）**互斥量**：为协调对一个共享资源的单独访问而设计，采用互斥对象机制，只有拥有互斥对象的线程才有访问公共资源的权限。因为互斥对象只有一个，所以可以保证公共资源不会被多个线程同时访问。互斥不仅能实现同一应用程序的公共资源安全共享，还能实现不同应用程序的公共资源安全共享。

（3）**信号量**：为控制一个具有有限数量的用户资源而设计。它允许同一时刻多个线程访问同一资源，但是需要控制同一时刻访问此资源的最大线程数量。

（4）**事件(信号)**：用来通知线程有一些事件已发生，从而启动后继任务的开始保持多线程同步，也可方便的实现多线程优先级的比较操作。

2、用户态和核心态

答：当一个任务（进程）执行系统调用而陷入内核代码中执行时，我们就称进程处于内核运行态（或简称为内核态）。此时处理器处于特权级最高的（0级）内核代码中执行。当进程处于内核态时，执行的内核代码会使用当前进程的内核栈。每个进程都有自己的内核栈。

当进程在执行用户自己的代码时，则称其处于用户运行态（用户态）。即此时处理器在特权级最低的（3级）用户代码中运行。当正在执行用户程序而突然被中断程序中断时，此时用户程序也可以象征性地称为处于进程的内核态。

3、进程的调度算法有哪些？解答见上文。

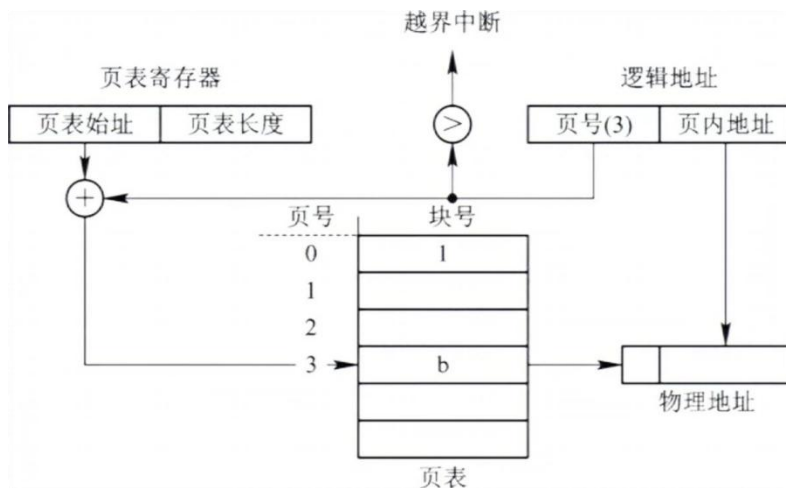
3.3 存储管理

3.3.1 主要功能

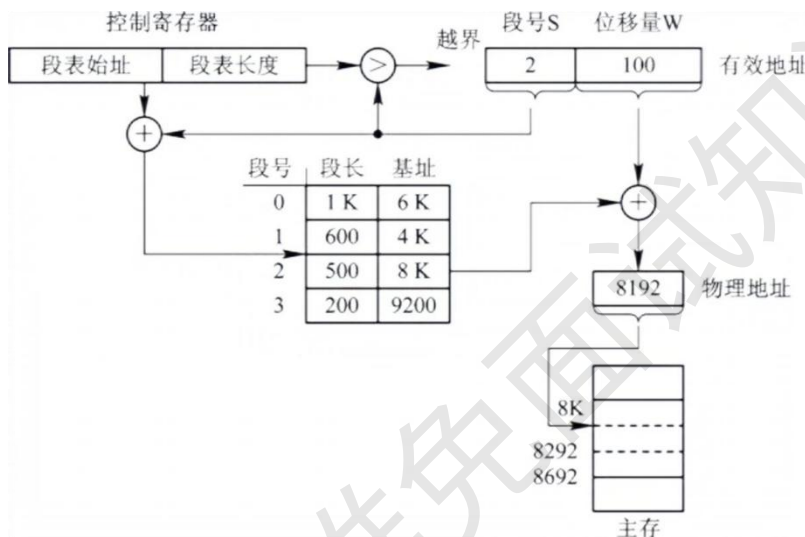
对存储空间进行分配和管理、存储器保护、地址转换、扩充主存容量(虚拟内存)。

3.3.2 段页式存储管理

（1）**页式存储管理**：将进程逻辑空间等分成若干大小的页面，相应的把物理内存空间分成与页面大小的物理块，以页面为单位把进程空间装进物理内存中分散的物理块。地址变换过程如下：



(2) **段式存储管理**：将进程逻辑空间分成若干段（不等分），段的长度由连续逻辑的长度决定。地址变换过程如下：



(3) 对比：

	页式存储	段式存储
地址空间	一维	二维
大小	固定且由系统决定	不固定，由其所完成的功能决定
内存碎片	没有外碎片,有内碎片	没有内碎片,有外碎片
信息共享	存储保护和共享受到限制	存储保护和信息的共享方便
目的	实现虚拟内存，从而获得更大的地址空间。	使程序和数据可以被划分为逻辑上独立的地址空间，并且有助于共享和保护。

3.3.3 虚拟内存

(1) **虚拟内存技术**：允许将一个作业分多次调入内存。实际是对**物理内存的扩充**，速度接近于内存，成本接近于辅存。

(2) **理论基础**：

①**时间局部性原理**：如果程序中某条指令一旦执行，不久后该指令可能再次执行；如果某数据被访问过，不久后该数据可能再次被访问。

②空间局部性原理：一旦程序访问了某个存储单元，在不久之后，其附近的存储单元也将被访问。

(3) 实现方法：建立在离散分配的存储管理方式的基础。一般分两种：请求分页系统、请求分段系统。

3.3.4 页面置换算法

(1) 最佳置换算法 OPT：即选择那些永不使用的，或者是在最长时间不再被访问的页面置换出去。它是一种理想化的算法，性能最好，但在实际上难于实现。

(2) 先进先出置换算法 FIFO：该算法总是淘汰最先进入内存的页面，即选择在内存中驻留时间最久的页面予以淘汰。

(3) 最近最久未使用置换算法 LRU：该算法是选择最近最久未使用的页面予以淘汰，系统在每个页面设置一个访问字段，用以记录这个页面自上次被访问以来所经历的时间 T，当要淘汰一个页面时，选择 T 最大的页面。

(4) Clock 置换算法：也叫**最近未用算法 NRU**。该算法为每个页面设置一位访问位，将内存中的所有页面都通过链接指针链成一个循环队列。当某页被访问时，其访问位置“1”。在选择一页淘汰时，就检查其访问位，如果是“0”，就选择该页换出；若为“1”，则重新置为“0”，暂不换出该页，在循环队列中检查下一个页面，直到访问位为“0”的页面为止。

(5) 最少使用置换算法 LFU：该算法选择最近时期使用最少的页面作为淘汰页。

常见问题：

分页存储与分段存储区别。解答见上文。

3.4 文件管理和设备管理

3.4.1 文件的逻辑结构

(1) 逻辑结构的文件类型：**有结构文件**（文本文件，文档，媒体文件）、**无结构文件**（二进制文件、链接库）。

(2) 顺序文件：按顺序放在存储介质中的文件，在逻辑文件当中存储效率最高，但不适合存储可变长文件。

(3) 索引文件：为解决可变长文件存储而发明，需要配合索引表存储。

3.4.2 文件共享与安全

(1) **文件共享**：文件共享是指某一个或某一部分文件可以让事先规定的某些用户共同使用。其目的为节省存储空间和进程间交换信息。

(2) **文件安全**：所谓文件安全，就是文件的保护问题。文件的保护是指文件本身不得被未经文件主授权的任何用户存取，而对于授权用户也只能在允许的存取权限内使用文件。文件保护可以通过**用户权验证**实现。

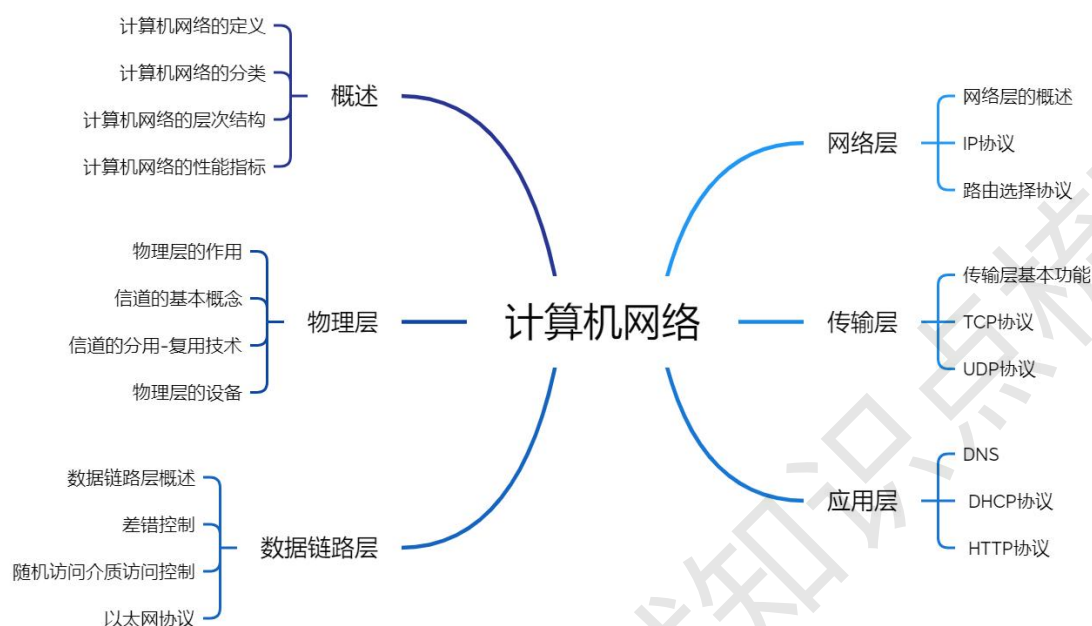
3.4.3 广义的 IO 设备

- (1) 按照使用特性分类：存储设备（内存、磁盘、U 盘）和交互 IO 设备（键盘、显示器、鼠标）
- (2) 按照信息交换分类：块设备（磁盘、SD 卡）和字符设备（打印机、shell 终端）
- (3) 按照设备共享属性分类：独占设备，共享设备，虚拟设备
- (4) 按照传输速率分类：低速设备，高速设备

3.4.4 SPOOLing 技术

虚拟设备技术，利用高速共享设备将低速的独享设备模拟为高速的共享设备，逻辑上，系统为每一个用户都分配了一台独立的高速独享设备。

4. 计算机网络



4.1 计算机网络概述

4.1.1 计算机网络的定义

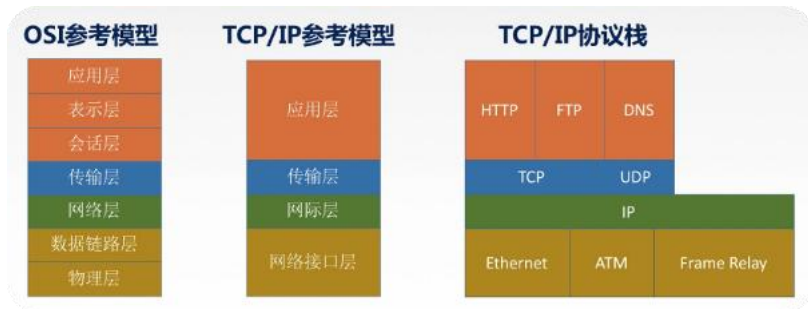
计算机网络是将一个分散的、具有独立功能的计算机系统，通过通信设备与线路连接，由功能完善的软件实现资源共享和信息传递的系统。计算机网络的两大核心功能为路由和转发。

4.1.2 计算机网络的分类

- (1) 按照网络的作用范围：广域网（WAN）、城域网（MAN）、局域网（LAN）。
- (2) 按照网络使用者：公用网络、专用网络。

4.1.3 计算机网络的层次结构

- (1) 层次设计的基本原则：各层之间是相互独立的；每一层需要有足够的灵活性；各层之间完全解耦。
- (2) OSI 参考模型与 TCP/IP 参考模型：前者为理论模型，后者为实践模型。



4.1.4 计算机网络的性能指标

- (1) 速率: $\text{bps} = \text{bit/s}$
- (2) 时延: 发送时延、传播时延、排队时延、处理时延
- (3) 往返时间 RTT: 数据报文在端到端通信中的来回一次的时间。

常见问题:

1、协议和服务之间的联系和区别

答: 首先, 协议是**对等层实体之间通信的规则集**。在协议的控制下, 两个对等实体间的通信使得本层能够向上一层提供服务。要实现本协议, 还需要使用下面一层所提供的服务。本层只能看见本层的服务, 看不见下层的协议, 即下层的协议对上层服务的实体而言是透明的。

其次, 协议是“水平的”, 即协议是控制对等实体之间通信的规则。但服务是“垂直的”, 即**服务是由下层向上层通过层间接口提供的**。

2、运输层协议与网络层协议的区别

答: 网络层协议负责的是提供主机间的逻辑通信, 运输层协议负责的是提供进程间的逻辑通信。

4.2 物理层

4.2.1 物理层的作用

连接不同的物理设备, 传输比特流。

4.2.2 信道的基本概念

(1) 定义: 信道是往一个方向传输信息的媒体, 一条通信电路包含一个发送信道和一个接受信道。

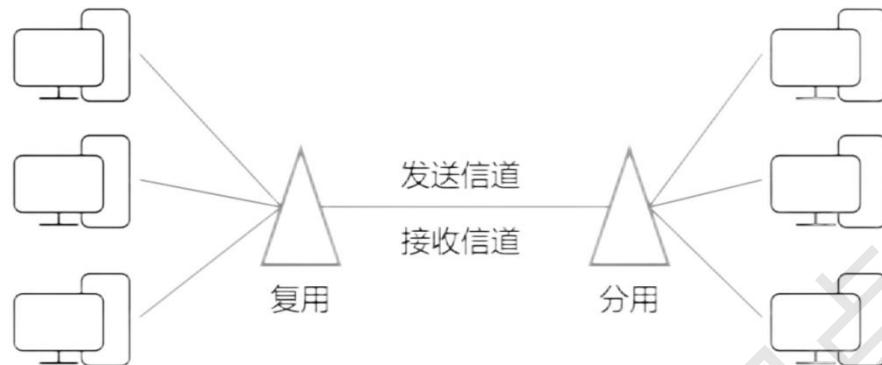
(2) 单工通信信道: 只能一个方向通信, 没有反方向反馈的信道。

(3) 半双工通信信道: 双方都可以发送和接受信息, 但不能同时发送也不能同时接收。

(4) 全双工通信信道: 双方都可以同时发送和接收。

4.2.3 信道的分用-复用技术

主要功能是**提升信道的利用率**。可分为频分复用、时分复用、波分复用、码分复用。



4.2.4 物理层的设备

- (1) 中继器：同一局域网的再生信号；两端口的网段必须同一协议；5-4-3 规程：10BASE-5 以太网中，最多串联 4 个中继器，5 段中只能有 3 个连接主机；
- (2) 集线器：同一局域网的再生、放大信号（多端口的中继器）；半双工，不能隔离冲突域也不能隔离广播域。

4.3 数据链路层

4.3.1 数据链路层概述

- (1) 数据链路层的功能：为网络层提供服务、链路管理、帧定界、帧同步、透明传输、差错控制、流量控制、随机访问介质访问控制。
- (2) 封装成帧：“帧”是数据链路层数据的基本单位，帧的结构如下图：



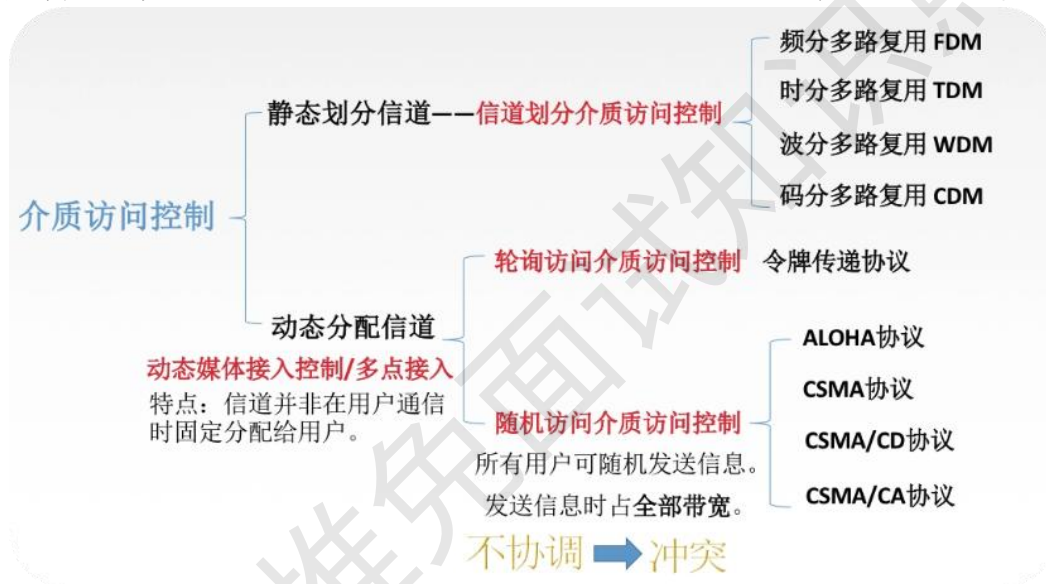
- (3) 数据交换：电路交换、报文交换、分组交换。
- (4) 透明传输：“透明”是指即使控制字符在帧数据中，但是要当做不存在去处理。原理如下图，即在控制字符前加上转义字符 ESC。
- (5) 最大传输单元 MTU：数据链路层的数据帧不是无限大的，数据帧长度受 MTU 限制。路径 MTU 由链路中 MTU 的最小值决定。

4.3.2 数据链路层的差错控制

- (1) 差错检测：奇偶校验码、循环冗余校验码 CRC。
- (2) **奇偶校验码-局限性**：当出错两位时，检测不到错误。
- (3) **循环冗余校验码**：根据传输或保存的数据而产生**固定位数校验码**。

4.3.3 随机访问介质访问控制

- (1) 随机接入系统 ALOHA：纯 ALOHA、时域 ALOHA 协议
- (2) CSMA（载波监听多路访问）：1 坚持、非坚持、P 坚持。
- (3) **CSMA/CD(碰撞检测)**：先听后发、边听边发、冲突停发、随机重发。适合**有线网**（局域网）。
- (4) **CSMA/CA（碰撞避免）**：包括预约信道、ACK、RTS、CTS，适用于**无线网**。并不能完全避免，只能尽量避免，在发送的过程中不检测冲突。



4.3.4 以太网协议

- (1) MAC 地址：每一个设备都拥有唯一的 MAC 地址，共 48 位，使用十六进制表示。



- (2) 以太网协议：是一种使用广泛的局域网技术，是一种应用于数据链路层的协议，使用以太网可以完成**相邻设备**的数据帧传输，数据格式如下：

目的地址	源地址	类型	数据	FCS
6	6	2	46~1500	4

4.4 网络层

4.4.1 网络层的概述

(1) 目的：实现两个端系统之间的数据透明传送，具体功能包括寻址和路由选择、连接的建立、保持和终止等。

(2) 数据交换技术：报文交换（基本上被分组所替代）。采用储存转发方式，数据交换单位是报文。

(3) 主要协议：

①IP 协议（Internet Protocol，因特网互联协议）。

②ICMP 协议（Internet Control Message Protocol，因特网控制报文协议）。

③ARP 协议（Address Resolution Protocol，地址解析协议）。

④RARP 协议（Reverse Address Resolution Protocol，逆地址解析协议）。

(4) 重要设备：路由器。

4.4.2 IP 协议

(1) IP 数据报的格式：



(2) 网络地址转换 NAT 技术：用于多个主机通过一个公有 IP 访问互联网的私有网络中，减缓了 IP 地址的消耗，但是增加了网络通信的复杂度。

(3) 分组转发：

①从数据报的首部提取目的主机的 IP 地址 D，得出目的网络地址为 N。

②若网络 N 与此路由器直接相连，则把数据报直接交付目的主机 D；否则是间接交付，执行③。

③若路由表中有目的地址为 D 的特定主机路由，则把数据报传送给路由表中所指明的下一跳路由器；否则，执行④。

④若路由表中有到达网络 N 的路由，则把数据报传送给路由表指明的下一跳路由器；否则，执行⑤。

⑤若路由表中有一个默认路由，则把数据报传送给路由表中所指明的默认路由器；否则，执行⑥。

⑥报告转发分组出错。

4.4.3 路由选择协议

(1) 内部网关协议 IGP: 一个自治系统内部使用的路由选择协议。有多种协议, 如 RIP 和 OSPF 协议。

(2) 外部网关协议 EGP: 一个自治系统的边界, 将路由选择信息传递到另一个自治系统中。目前使用的就是 BGP

4.5 传输层

4.5.1 传输层基本功能

(1) 基本服务: 将主机间交付扩展到进程间交付, 通过复用和分用实现。

(2) 复用: 发送方传输层将套接字标识置于报文段中, 交给网络层

(3) 分用: 接收方传输层根据报文段中的套接字标识, 将报文段交付到正确的套接字

4.5.2 TCP 协议

(1) TCP 协议结构



(2) TCP 协议的可靠传输

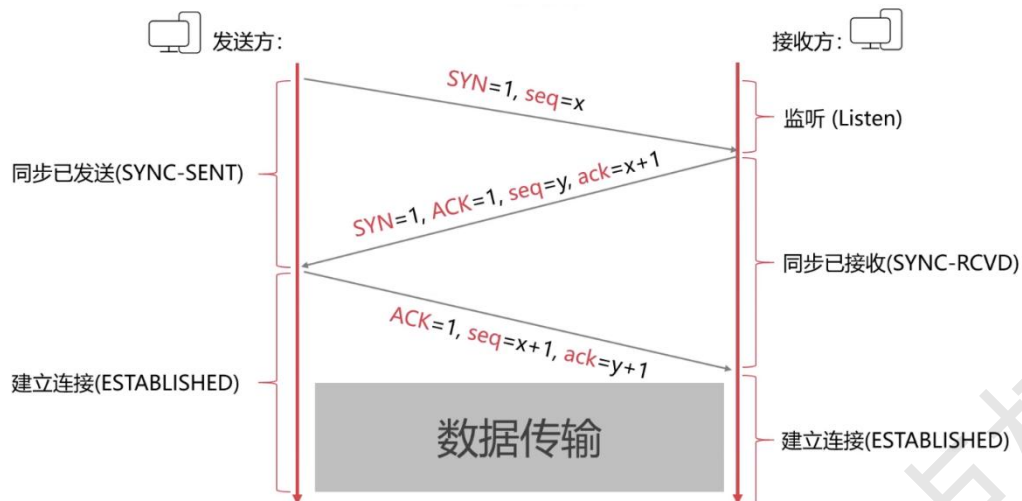
①停止等待协议: 是最简单的可靠传输协议, 但是该协议对信道的利用率不高。

②连续 ARQ 协议(自动重传请求): 滑动窗口+累计确认, 大幅提高了信道的利用率。

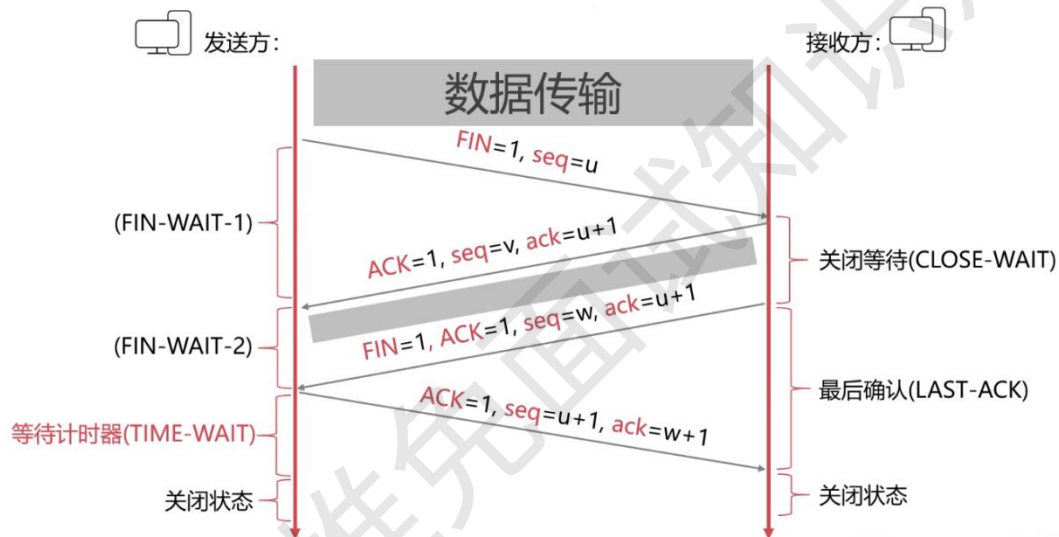
TCP 协议基于连续 ARQ 协议实现可靠传输, 会重复传输部分已经成功接收的字节。

(3) TCP 协议的拥塞控制: 拥塞控制通过慢启动、拥塞避免、快重传、快恢复实现。

(4) TCP 连接的三次握手

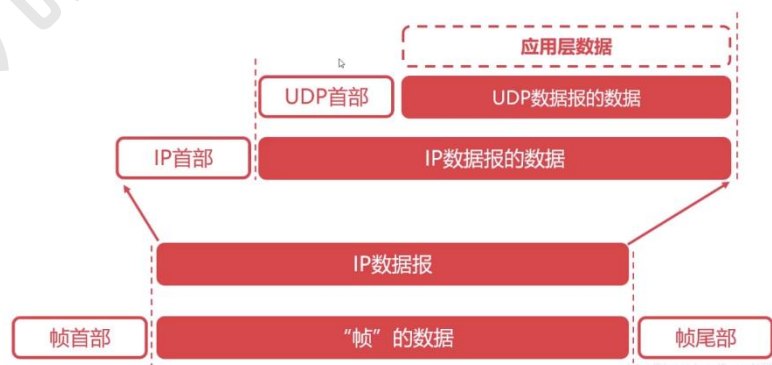


(5) TCP 协议的四次挥手



4.5.3 UDP 协议

(1) UDP 协议结构



(2) UDP 协议与 TCP 协议区别

- ① TCP 提供面向连接的传输；UDP 提供无连接的传输。

②TCP 提供可靠的传输（有序，无差错，不丢失，不重复）；UDP 提供不可靠的传输。

③TCP 面向字节流的传输，因此它能将信息分割成组，并在接收端将其重组；UDP 是面向数据报的传输，没有分组开销。

④TCP 提供拥塞控制和流量控制机制；UDP 不提供拥塞控制和流量控制机制。

⑤TCP 只能是点对点的（一对一）。UDP 支持一对一、一对多、多对一和多对多的交互通信。

常见问题

1、拥塞控制与流量控制的区别

答：流量控制考虑点对点的通信量的控制，而拥塞控制考虑整个网络，是全局性的考虑。

2、为什么需要三次握手？

答：防止失效的连接请求报文段被服务端接收，从而产生错误。

4.6 应用层

4.6.1 DNS

（1）DNS：解决 IP 地址复杂难以记忆的问题，存储并完成自己所管辖范围内主机的域名到 IP 地址的映射。

（2）域名解析的顺序：

①浏览器缓存

②找本机的 hosts 文件

③路由缓存

④找 DNS 服务器（本地域名、顶级域名、根域名）->迭代解析、递归查询。

4.6.2 DHCP 协议

（1）定义：是一个局域网协议，是应用 UDP 协议的应用层协议。

（2）作用：为临时接入局域网的用户自动分配 IP 地址。

4.6.3 HTTP 协议

HTTP 是面向事务的应用层协议。定义了浏览器怎样向万维网服务器请求万维网文档，以及服务器怎样把文档传送给浏览器。HTTP 是万维网上能够可靠地交换文件的重要基础。

常见问题

说下浏览器请求一个网址的过程？

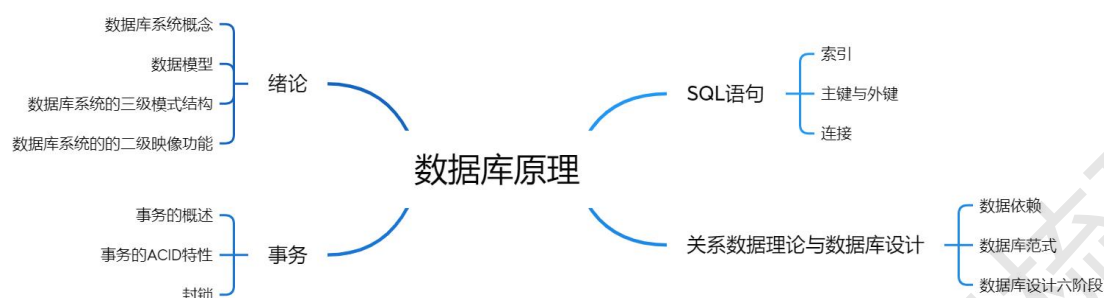
答：（1）首先是解析域名，DNS 进行，获得 IP 地址。然后端口号发送端随机分配，服务器是固定的 80 端口。

（2）知道 IP 和端口号之后就可以建立 3 次握手建立 TCP 连接。

- (3) 发送 HTTP 请求报文。
- (4) 服务器处理浏览器发来的请求，返回要显示的页面 `html` 信息。
- (5) 传输结束，4 次握手释放连接。
- (6) 显示页面。

计算机类推免面试知识点梳理

5. 数据库原理



5.1 绪论

5.1.1 数据库系统概述

- (1) **Data:** 数据，是数据库中存储的基本对象，是描述事物的符号记录。
- (2) **Database:** 数据库，是长期储存在计算机内、有组织的、可共享的大量数据的集合。
- (3) **DBMS:** 数据库管理系统，是位于用户与操作系统之间的一层数据管理软件，用于科学地组织、存储和管理数据、高效地获取和维护数据。
- (4) **DBS:** 数据库系统，指在计算机系统中引入数据库后的系统，一般由数据库、数据库管理系统、应用系统、数据库管理员（DBA）构成。
- (5) **实体和属性:** 客观存在并可相互区别的事物称为实体。实体所具有的某一特性称为属性。
- (6) **E-R 图:** 即实体-关系图，用于描述现实世界的事物及其相互关系，是数据库概念模型设计的主要工具。

5.1.2 数据模型

- (1) 定义: **Data Model**，是用来抽象、表示和处理现实世界中的数据和信息的工具，是对现实世界的模拟，是数据库系统的核心和基础；其组成元素有数据结构、数据操作和完整性约束。
- (2) **概念模型:** 也称信息模型，是按用户的观点来对数据和信息建模，主要用于数据库设计。
- (3) **逻辑模型:** 是按计算机系统的观点对数据建模，用于 **DBMS** 实现。
- (4) **物理模型:** 是对数据最底层的抽象，描述数据在系统内部的表示方式和存取方法，在磁盘或磁带上的存储方式和存取方法，是面向计算机系统的。

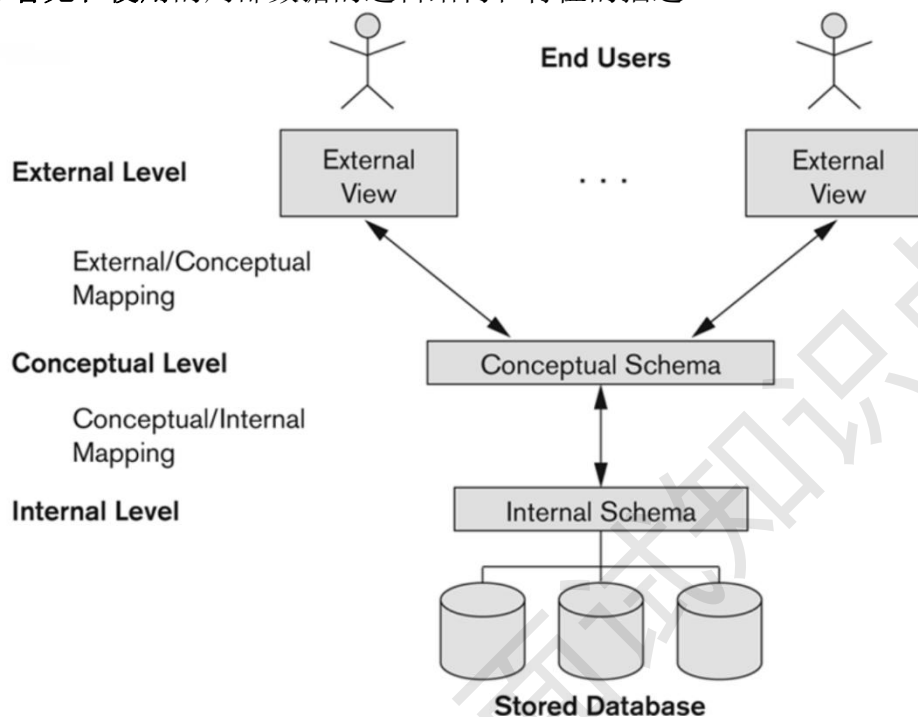
5.1.3 数据库系统的三级模式结构

- (1) 数据库模型: 是对数据库中全体数据的**逻辑结构**（数据项的名字、类型、取值范围等）和**特征**（数据之间的联系以及数据有关的安全性、完整性要求）的描述。

(2) **内模式 (Internal Schema)**：(也称存储模式。)数据物理结构和储存方式的描述，是数据在**数据库内部**的表示方式。

(3) **概念模式 (Conceptual Schema)**：(也称全局模式。)有时简称“模式”。是对数据库中全体数据的**逻辑结构和特征**的描述。

(4) **外模式 (External Schema)**：(也称子模式或者用户模式)数据库用户能够**看见和使用的**局部数据的逻辑结构和特征的描述。



5.1.4 数据库系统的二级映像功能

(1) **二级映像**：为了在系统内部实现这三个抽象层次的联系和转换，数据库管理系统在三级模式之间提供了两层映像。这两层映像保证了数据库系统中的数据能够具有较高的**逻辑独立性和物理独立性**。

(2) **外模式/模式映像**：当模式改变时，由数据库管理员对各个外模式/模式的映像作相应改变，可以保证外模式保持不变。应用程序是依据数据的外模式编写的，从而应用程序不必改变，保证了数据与程序的逻辑独立性，简称**数据的逻辑独立性**。

(3) **模式/内模式映像**：当数据库的存储结构改变时，由数据库管理员对模式/内模式映像做相应改变，可以使模式保持不变，从而应用程序也不必改变，保证了数据与程序的物理独立性，简称**数据的物理独立性**。

5.2 事务

5.2.1 事务的概述

(1) **定义**：数据库事务通常包含了一个序列的对数据库的读/写操作（一个单元的一系列 SQL 语句的集合）。

(2) **目的**：

①系统错误恢复：为数据库操作序列提供了一个从失败中恢复到正常状态的方法，同时提供了数据库即使在异常状态下仍能保持一致性的方法

②并发数据库访问：当多个应用程序在并发访问数据库时，可以在这些应用程序之间提供一个隔离方法，以防止彼此的操作互相干扰。

5.2.2 事务的 ACID 特性

(1) A, atomacity 原子性：指整个数据库事务是不可分割的工作单位。只有事务中所有的数据库操作都执行成功，整个事务的执行才算成功。事务中任何一个 sql 语句执行失败，那么已经执行成功的 sql 语句也必须撤销，数据库状态应该退回到执行事务前的状态。

(2) C, consistency 一致性：事务将数据库从一种一致状态转变为下一种一致状态。也就是说，事务在完成时，必须使所有的数据都保持一致状态(各种 constraint 不被破坏)。

(3) I, isolation 隔离性：事务的隔离性要求每个读写事务的对象与其它事务的操作对象能相互分离，即该事务提交前对其它事务都不可见，这通常使用锁来实现多个事务并发执行时，一个事务的执行不应影响其他事务的执行。

(4) D, durability 持久性：事务完成之后，它对于系统的影响是永久性的。该修改即使出现致命的系统故障也将一直保持。

5.2.3 封锁

(1) 定义：封锁就是事务 T 在对某个数据对象例如表、记录等操作之前，先向系统发出请求，对其加锁。

(2) 数据库系统提供两种锁：

①排他锁（写锁）：若事务 T 对数据对象 A 加写锁，则只允许 T 读取和修改 A，其他事务都不能再对 A 加任何类型的锁，直到 T 释放 A 上的锁为止。

②共享锁（读锁）：若事务 T 对数据对象 A 加读锁，则只允许 T 可以读取但不能修改 A，其他事务只能再对 A 加读锁，而不能加写锁，直到 T 释放 A 上的读锁为止。

(3) 封锁协议：使用读锁和写锁时，需要约定一定的规则。比如：何时申请、持续时间、何时释放等。这些规则被称为封锁协议。针对不同的事务隔离级别，有四种封锁协议。

常见问题：

1、数据库的乐观锁和悲观锁是什么？

答：数据库管理系统（DBMS）中的并发控制的任务是确保在多个事务同时存取数据库中同一数据时不破坏事务的隔离性和统一性以及数据库的统一性。

乐观并发控制(乐观锁)和悲观并发控制（悲观锁）是并发控制主要采用的技术手段。

悲观锁：假定会发生并发冲突，屏蔽一切可能违反数据完整性的操作

乐观锁：假设不会发生并发冲突，只在提交操作时检查是否违反数据完整性。

2、简介事务，以及事务的 ACID 特性。解答见上文。

5.3 SQL 语句

5.3.1 索引

(1) 定义：类似于字典，数据库中的索引是用于**提高数据库表数据访问速度的数据库对象**。

(2) 分类：

①聚集索引：表数据按照**索引的顺序**来存储的。对于聚集索引，叶子结点即存储了真实的数据行，不再有另外单独的数据页。在一张表上只能创建一个聚集索引。

②非聚集索引：表数据**存储顺序与索引顺序无关**。对于非聚集索引，叶子结点包含索引字段值及指向数据页数据行的逻辑指针，该层紧邻数据页，其行数量与数据表行数据量一致。

5.3.2 主键与外键

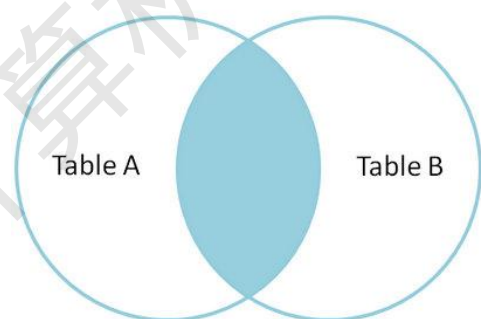
(1) 作用：维护关系数据库的完整性。

(2) 主键、外键、索引辨析

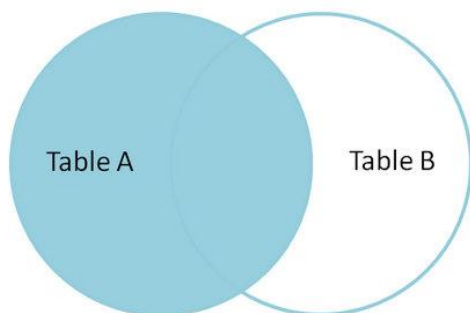
	定义	作用	个数
主键	唯一标识一条记录，不能有重复，不允许为空。	用来保证数据完整性	主键只能有一个。
外键	表的外键为另一个表的主键，外键可以有重复，可以为空。	用来和其他表建立联系	一个表可以有多个外键。
索引	可以为重复值，可以作为一个空值。	用来提高查询排序的速度	一个表可以有多个唯一索引。

5.3.3 连接

(1) INNER JOIN（内连接，或等值连接）：**n** 产生同时符合左表和右表的一组数据。

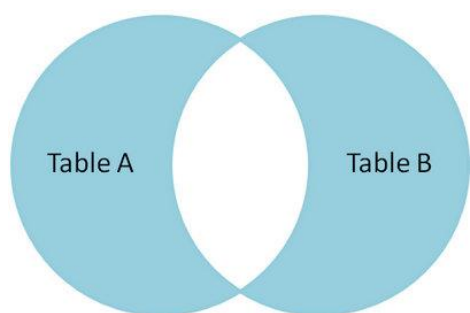


(2) LEFT JOIN（左连接）：取得左表完全记录，即是右表并无对应匹配记录。



(3) **RIGHT JOIN** (右连接)：与 **LEFT JOIN** 相反，取得右表完全记录，即是左表并无匹配对应记录。

(4) **CROSS JOIN** (交叉连接)：交叉连接，得到的结果是两个表的乘积，即笛卡尔积。



常见问题

1、索引的优缺点

答：(1) 索引加快数据库的检索速度。

(2) 索引降低了插入、删除、修改等维护任务的速度。

(3) 唯一索引可以确保每一行数据的唯一性。

(4) 通过使用索引，可以在查询的过程中使用优化隐藏器，提高系统的性能

(5) 索引需要占物理和数据空间。

2、索引设计原则

答：(1) 表的某个字段值得离散度越高，该字段越适合选作索引的关键字。

(2) 占用存储空间少的字段更适合选作索引的关键字。

(3) 存储空间固定的字段更适合选作索引的关键字。

(3) **Where** 子句中经常使用的字段应该创建索引，分组字段或者排序字段应该创建索引，两个表的连接字段应该创建索引。

(4) 更新频繁的字段不适合创建索引，不会出现在 **where** 子句中的字段不应该创建索引。

5.4 关系数据理论与数据库设计

5.4.1 数据依赖

数据依赖是一个关系内部属性与属性之间的一种约束关系。有多种类型的数据依赖，其中最重要的是函数依赖和多值依赖。

5.4.2 数据库范式

(1) 1NF (First Normal Form)：当且仅当所有域只包含原子值，即每个分量都是不可再分的数据项，则称实体 E 满足第一范式。

(2) 2NF (Second Normal Form)：当且仅当实体 E 满足第一范式，且每一个非键属性完全依赖主键时，满足第二范式。

(3) 3NF (Third Normal Form)：当且仅当实体 E 是第二范式 (2NF)，且 E 中没有非主属性传递依赖时，满足第三范式。

5.4.3 数据库设计六阶段

(1) **需求分析**：准确了解和分析用户需求（包括数据和处理）。

(2) **概念结构设计**：通过对用户需求进行综合、归纳与抽象，形成一个独立于具体 DBMS 的概念模型。

(3) **逻辑结构设计**：将概念结构转换为某个 DBMS 所支持的数据模型，并对其进行优化。

(4) **物理结构设计**：为逻辑数据模型选取一个最适合应用环境的物理结构（包括存储结构和存取方法）。

(5) **数据库实施**：根据逻辑设计和物理设计的结果建立数据库，编制与调试应用程序，组织数据入库并试运行。

(6) **数据库运行和维护**：在数据库投入正式使用后不断地对其进行评价、调整与修改。

常见问题

数据库系统的基本特点

答：(1) 数据结构化。

(2) 数据的共享性高、冗余度低且易扩充

(1) 数据独立性高。包括物理独立性和逻辑独立性。

(2) 数据由数据库管理系统统一管理和控制。